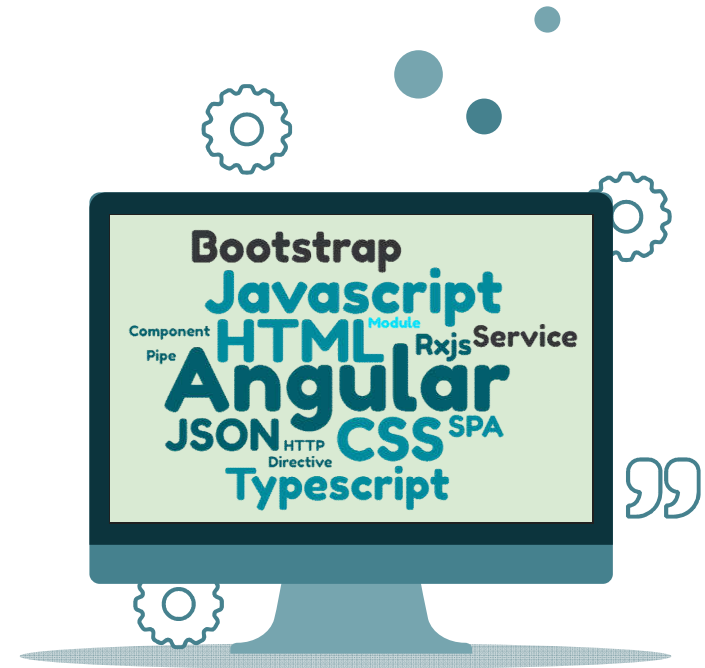


04

Fondamentaux d'Angular



Chapitre 4 Fondamentaux d'Angular

01

Composants

02

Data Binding

03

Bootstrap

04

Services

05

Directives

06

Pipes

07

Interraction entre les
composants

08

Atelier 2

01

Composants



Présentation de l'application

- Une application avec deux Fonctionnalités
 - Voir un about sur un profil et laisser un commentaire
 - Gérer une liste de contacts
- Maquette

first-project About Contacts

About Component

- Name : Anouar
- Email : medanouar.dahdeh@gmail.com
- Tel : 12345678

Message

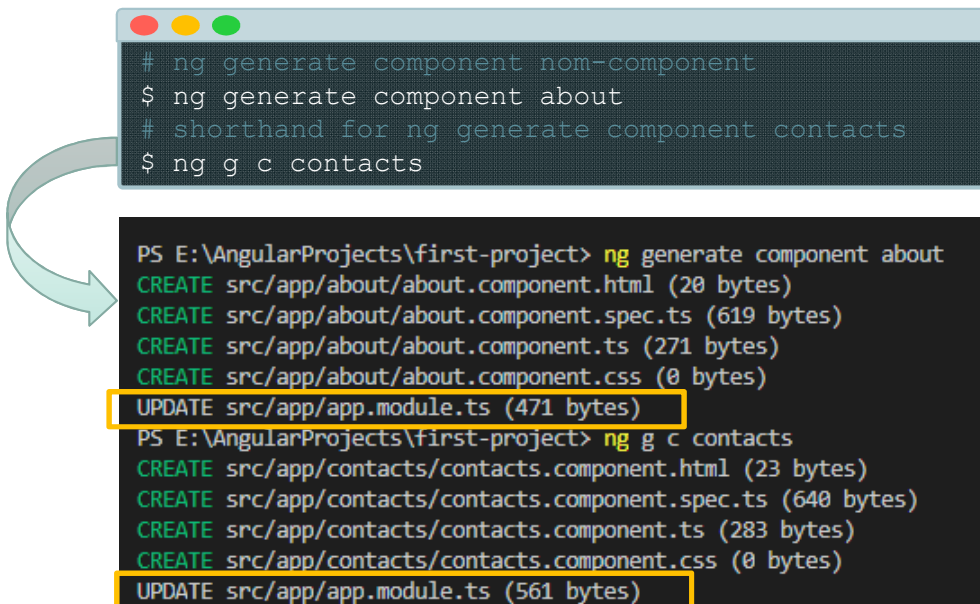
Your message

Add Comment

No Comments

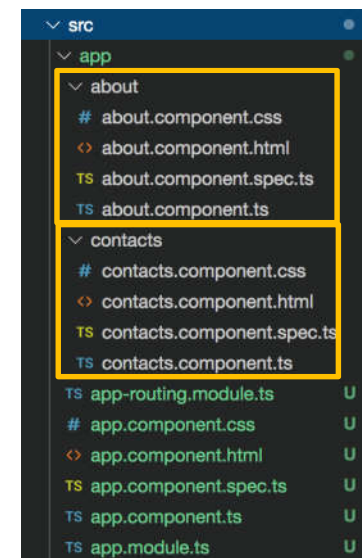
Créer un composant

- Créer deux nouveaux composants (About et Contacts)



```
# ng generate component nom-component
$ ng generate component about
# shorthand for ng generate component contacts
$ ng g c contacts
```

PS E:\AngularProjects\first-project> ng generate component about
CREATE src/app/about/about.component.html (20 bytes)
CREATE src/app/about/about.component.spec.ts (619 bytes)
CREATE src/app/about/about.component.ts (271 bytes)
CREATE src/app/about/about.component.css (0 bytes)
UPDATE src/app/app.module.ts (471 bytes)
PS E:\AngularProjects\first-project> ng g c contacts
CREATE src/app/contacts/contacts.component.html (23 bytes)
CREATE src/app/contacts/contacts.component.spec.ts (640 bytes)
CREATE src/app/contacts/contacts.component.ts (283 bytes)
CREATE src/app/contacts/contacts.component.css (0 bytes)
UPDATE src/app/app.module.ts (561 bytes)



Déclaration des composants

- Pour utiliser un composant, ce dernier doit être déclaré dans le module,
- La commande `ng g c <nom-composant>`, met à jour automatique le fichier `app.modules.ts`

```
CREATE src/app/about/about.component.css (0 bytes)  
UPDATE src/app/app.module.ts (471 bytes)
```

```
CREATE src/app/contacts/contacts.component.css (0 bytes)  
UPDATE src/app/app.module.ts (561 bytes)
```



```
TS app.module.ts x  
src > app > TS app.module.ts > ...  
1  import { BrowserModule } from '@angular/platform-browser';  
2  import { NgModule } from '@angular/core';  
3  
4  import { AppRoutingModule } from './app-routing.module';  
5  import { AppComponent } from './app.component';  
6  import { AboutComponent } from './about/about.component';  
7  import { ContactsComponent } from './contacts/contacts.component';  
8  
9  @NgModule({  
10   declarations: [  
11     AppComponent,  
12     AboutComponent,  
13     ContactsComponent  
14   ],  
15   imports: [  
16     BrowserModule,  
17     AppRoutingModule  
18   ],  
19   providers: [],  
20   bootstrap: [AppComponent]  
21 })  
22 export class AppModule { }  
23
```

monamedanoudi.danden@tsb.ucar.tn

Contenu d'un composant

```
TS about.component.ts ×
src > app > about > TS about.component.ts > ...
1  import { Component, OnInit } from '@angular/core';
2
3  @Component({
4    selector: 'app-about',
5    templateUrl: './about.component.html',
6    styleUrls: ['./about.component.css']
7  })
8  export class AboutComponent implements OnInit {
9
10   constructor() { }
11
12   ngOnInit() {
13   }
```

```
<> about.component.html ×
src > app > about > <> about.component.html > ...
1  <p>about works!</p>
2  |
```

```
# about.component.css ×
src > app > about > # about.component.css
1  |
```

Constructor vs. ngOnInit

- **constructor()**: appelé en 1^{er} avant la méthode **ngOnInit()**
- **ngOnInit()**: appelée après le constructeur

Généralement, **ngOnInit** est utilisé pour toutes les initialisations / déclarations. Le constructeur ne doit être utilisé que pour initialiser les attributs de la classe mais ne doit pas faire de "travail" réel.

Le décorateur @Component(1/2)

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
1 import { Component, OnInit } from '@angular/core';
2
3 @Component({
4   selector: 'app-about',
5   templateUrl: './about.component.html',
6   styleUrls: ['./about.component.css']
7 })
8 export class AboutComponent implements OnInit {
9
10  constructor() { }
11
12  ngOnInit() {
13  }
```

```
TS contacts.component.ts X
src > app > contacts > TS contacts.component.ts > ...
1 import { Component, OnInit } from '@angular/core';
2
3 @Component({
4   selector: '[app-contacts]',
5   templateUrl: './contacts.component.html',
6   styleUrls: ['./contacts.component.css']
7 })
8 export class ContactsComponent implements OnInit {
9
10  constructor() { }
11
12  ngOnInit(): void {
13  }
```

@Component

- **Un composant** est une classe qui possède le décorateur @Component
- **Ce décorateur** possède les propriétés suivantes :
 - **selector**: indique la déclaration qui permet d'insérer le composant dans le document HTML. Cette déclaration peut être :
 - Le nom de la balise associé à ce composant
 - selector : 'app-about'
 - Dans ce cas le composant sera inséré par : `<app-about> <app-about>`
 - Le nom de l'attribut associé à ce composant
 - selector : ' [app-contacts] '
 - Dans ce cas le composant sera inséré par : `<div app-contacts>...<div>`
 - Le nom de la classe associé à ce composant
 - selector : '.app-about'
 - Dans ce cas le composant sera inséré par : `<div class="app-about">...<div>`

Le décorateur @Component(2/2)

```
TS contacts.component.ts X
src > app > contacts > TS contacts.component.ts > ...
1  import { Component, OnInit } from '@angular/core';
2
3  @Component({
4    selector: 'app-contacts',
5    template: '<p>contacts works!</p>',
6    styleUrls: ['./contacts.component.css']
7  })
8  export class ContactsComponent implements OnInit {
9
10   constructor() { }
11
12   ngOnInit(): void {
13   }
14
15 }
```

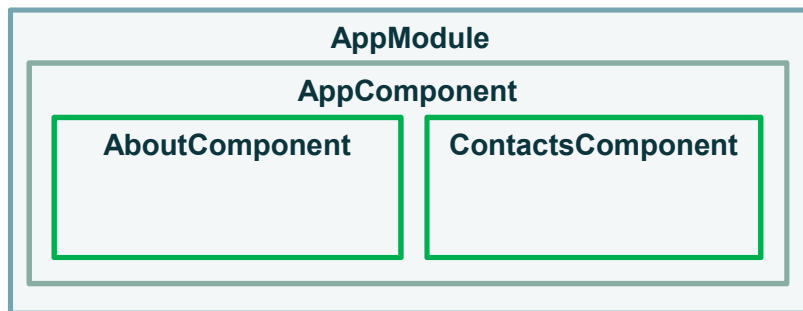
```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
1  import { Component, OnInit } from '@angular/core';
2
3  @Component({
4    selector: 'app-about',
5    templateUrl: './about.component.html',
6    styleUrls: ['./about.component.css']
7  })
8  export class AboutComponent implements OnInit {
9
10   constructor() { }
11
12   ngOnInit() {
13   }
14 }
```

@Component

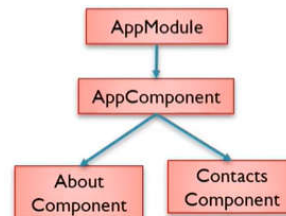
- **template ou templateUrl**
 - **template**: permet de définir à l'intérieur du décorateur le code HTML
 - **templateUrl**: permet d'associer un fichier externe HTML contenant la structure de la vue du composant
- **Style Urls**: permet de spécifier les feuilles de styles CSS associées à ce composant

Intégration d'un composant

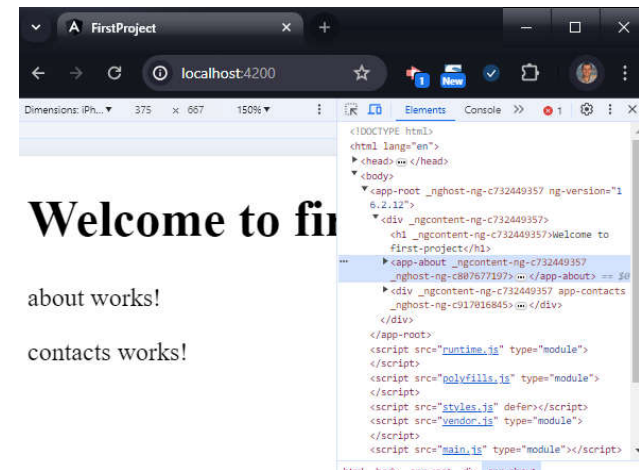
- Un composant peut être inséré dans n'importe quelle partie HTML de l'application en utilisant son **sélecteur** associé.
- Dans cet exemple, les deux composants générés sont insérés à l'intérieur du composant racine **AppComponent**



©MAD



```
app.component.html M X
src > app > app.component.html > ...
Go to component
1 <div>
2   <h1>Welcome to {{ title }}</h1>
3   <app-about></app-about>
4   <div app-contacts></div>
5 </div>
```



mohamedanouer.dahdeh@fsb.ucar.tn

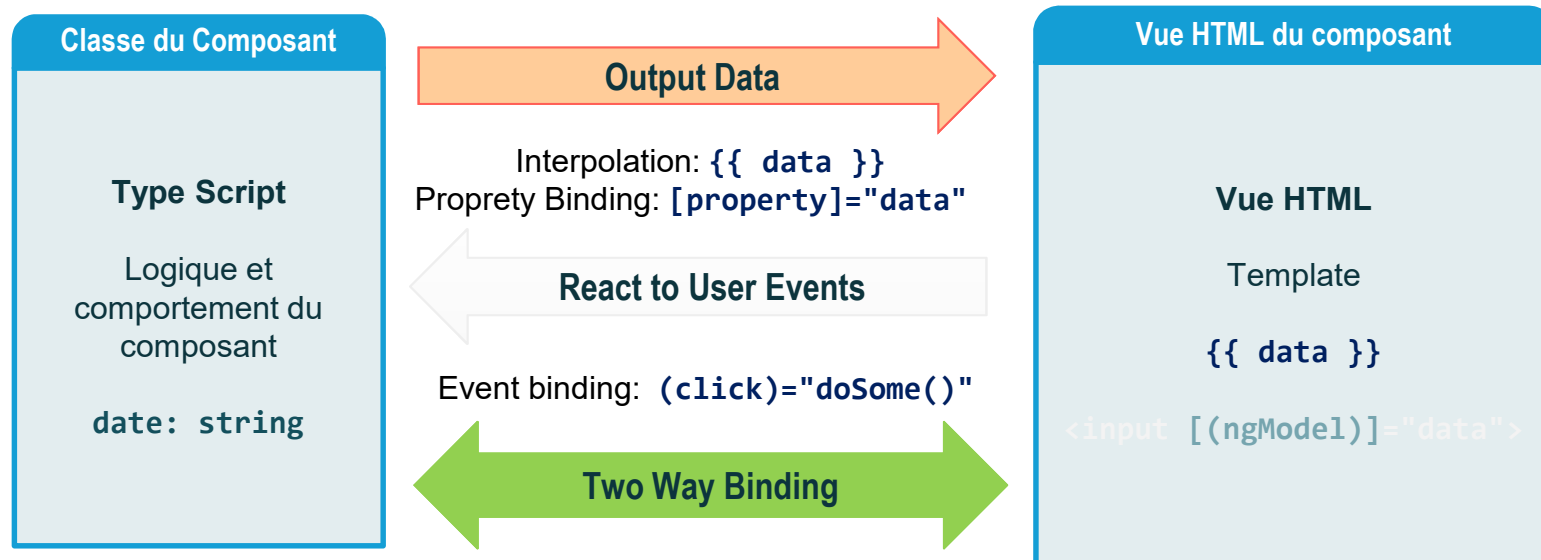
02

Data Binding



Data Binding : Interactions TypeScript & Template HTML

- Pour insérer dynamiquement des données de l'applications dans les vues des composants, Angular définit des techniques pour assurer la liaison des données.
(Data Binding = Communication)



Data Binding : One-way data binding

- Utiliser string interpolation `{{}}` et property binding `[]` syntaxe pour le one-way data binding avec les données circulant du composant vers le Template.

```
1 <!-- Interpolation-->
2 <!--The simplest way to insert data dynamically into your components is through text interpolation, using the
   {{myVariable}} syntax.-->
3 <h1>{{title}}</h1> <!-- This interpolation binding presents the component's title property value inside the
   HTML header tag.-->
4
5 <!--Property binding-->
6 <!--To bind an HTML element to a component's property, enclose it in square brackets [].-->
7 <!--The brackets,[], cause Angular to evaluate the right-hand side of the assignment as a dynamic expression.--
   >
8 <a [href]="url">Link</a>
9 <button [disabled]="isUnchanged">Disabled Button</button>
```

Uniquement de TypeScript vers le template HTML

Data Binding : One-way data binding

- Class and style binding

```
1 <!--Class binding-->
2 <!--To create a single class binding, use the prefix class followed by a dot and the name of the CSS class-->
3 <p [class.my-class-1]="isWarning"></p> <!--Angular adds the class when the bound expression, isWarning, is
   truthy and it removes the class when the expression is falsy.-->
4 <!--Multiple classes can also be bound with the [class] syntax:-->
5 <!-- classExpression = "my-class-1 my-class-2 my-class-3" -->
6 <!-- classExpression = {my-class-1: true, my-class-2: false} -->
7 <!-- classExpression = ['my-class-1', 'my-class-2'] -->
8 <p [class]="classExpression"></p>
9
10 <!--Style binding-->
11 <!--To create a single style binding, use the prefix style followed by a dot and the name of the CSS style
   property-->
12 <!-- Style properties can be written in dash-case or camelCase -->
13 <nav [style.backgroundColor]="expression"></nav>
14 <nav [style.backgroundColor]="expression"></nav>
15 <!--To toggle multiple styles, bind to the [style] attribute:-->
16 <!-- styleExpression = "width: 100px; height: 100px; background-color: red;" -->
17 <!-- styleExpression = {width: '100px', height: '100px', backgroundColor: 'red'} -->
18 <p [style]="styleExpression"></p>
19
```

Data Binding : One-way data binding

- **Event binding** `()` permet d'écouter et de répondre aux actions de l'utilisateur telles que les mouvements de souris, les clics ou un événement custom émis par un composant enfant.
- Syntaxe : `(event)="methode"`: Appelle une méthode quand un événement du DOM arrive

```
1 <!--Event binding-->  
2 <!--To bind to an event you use the Angular event binding syntax ().-->  
3 <button (click)="delete()">Delete</button>
```

Uniquement du template HTML vers le TypeScript

Data Binding : Tow-way data binding

- La directive **NgModel** vous permet de lier la valeur d'un champ de formulaire à une propriété du composant. Il s'agit d'une liaison **bidirectionnelle (Two-way binding)** : la propriété est mise à jour lorsque le contenu du champ change (typiquement par l'utilisateur) et vice versa. La syntaxe pour la liaison de données bidirectionnelle est `[(ngModel)]`
- Syntaxe : `[(ngModel)]="valeur"`

```
1 <!--Tow way binding.-->
2 <label>
3   What is your name ?
4   <input [(ngModel)]="name">
5 </label>
6
7 <p>Hello {{ name }} !</p>
```

What is your name ?

Hello Adam !

Dans les deux sens (Bidirectionnel)



Exemples de Data Binding

#01

Entant que
utilisateur je veux
voir un About

#02

Entant que
utilisateur je veux
ajouter des
commentaires

#03

Entant que
utilisateur je veux
voir des
commentaires

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
8 export class AboutComponent implements OnInit {
9
10   info = {
11     name: "Anouar",
12     email: "medanouer.dahdeh@gmail.com",
13     tel: "12345678"
14   };
15   comments = [];
16   comment = { date: null, message: '' };
17   newComment = false;
18   constructor() { }
19
20   ngOnInit(): void {
21   }
22   addComment() {
23     if (this.comment.message != '') {
24       this.comment.date = new Date();
25       this.comments.push({
26         date: this.comment.date,
27         message: this.comment.message
28       });
29       this.comment.message = '';
30     }
31   }
32 }
```

```
about.component.html X
src > app > about > about.component.html > ...
1 <ul>
2   <li>Name : {{info.name}}</li>
3   <li>Email : {{info.email}}</li>
4   <li>Tel : {{info.tel}}</li>
5 </ul>
6 <div>
7   <input
8     type="text"
9     [(ngModel)]="comment.message">
10
11   <button
12     (click)="addComment()"
13     [disabled]="newComment">
14     Add Comment
15   </button>
16 </div>
17 <div *ngIf="comments.length>0 ; else noComments">
18   <h3>Messages</h3>
19   <ul>
20     <li *ngFor="let c of comments">
21       {{c.date | date}} : {{c.message}}
22     </li>
23   </ul>
24 </div>
25 <ng-template #noComments>
26   <p>No Comments</p>
27 </ng-template>
```

Data Binding : OutPut Data

The diagram illustrates data binding in Angular. On the left, the `about.component.ts` file defines a class `AboutComponent` with properties `name`, `email`, and `tel`, and a `comments` array. In the middle, the `about.component.html` file shows the HTML template with interpolation for the user information and property binding for the `disabled` attribute of the `Add Comment` button. On the right, a browser screenshot shows the rendered output: the user's details and a list of comments. Arrows indicate the flow of data from the TypeScript code to the HTML template and then to the browser output.

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
8 export class AboutComponent implements OnInit {
9
10   info = {
11     name: "Anouar",
12     email: "medanouar.dahdeh@gmail.com",
13     tel: "12345678"
14   };
15   comments = [];
16   comment = { date: null, message: '' };
17   newComment = false;
18   constructor() { }
19
20   ngOnInit(): void {
21   }
22   addComment() {
23     if (this.comment.message != '') {
24       this.comment.date = new Date();
25       this.comments.push({
26         date: this.comment.date,
27         message: this.comment.message
28       });
29       this.comment.message = '';
30     }
31   }
32 }
```

```
<? about.component.html X
src > app > about > <? about.component.html > div
1 <ul>
2   <li>Name : {{info.name}} </li>
3   <li>Email : {{info.email}} </li>
4   <li>Tel : {{info.tel}} </li>
5 </ul>
6 <div>
7   <input
8     type="text"
9     [(ngModel)]="comment.message">
10
11   <button
12     (click)="addComment()"
13     [disabled]="newComment">
14     Add Comment
15   </button>
16 </div>
17 <div *ngIf="comments.length>0 ; else noComments">
18   <h3>Messages</h3>
19   <ul>
20     <li *ngFor="let c of comments">
21       {{c.date | date}} : {{c.message}}
22     </li>
23   </ul>
24 </div>
25 <ng-template #noComments>
26   <p>No Comments</p>
27 </ng-template>
28 </div>
```

interpolation

Property binding

Welcome to first-project

- Name : Anouar
- Email : medanouar.dahdeh@gmail.com
- Tel : 12345678

Add Comment

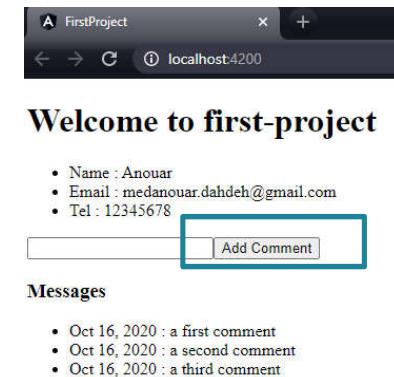
Messages

- Oct 16, 2020 : a first comment
- Oct 16, 2020 : a second comment
- Oct 16, 2020 : a third comment

Data Binding : React to user event

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
8   export class AboutComponent implements OnInit {
9
10    info = {
11      name: "Anouar",
12      email: "medanouar.dahdeh@gmail.com",
13      tel: "12345678"
14    };
15    comments = [];
16    comment = { date: null, message: '' };
17    newComment = false;
18    constructor() { }
19
20    ngOnInit(): void {
21    }
22
23    addComment() {
24      if (this.comment.message != '') {
25        this.comment.date = new Date();
26        this.comments.push({
27          date: this.comment.date,
28          message: this.comment.message
29        });
30        this.comment.message = '';
31      }
32    }
33  }
```

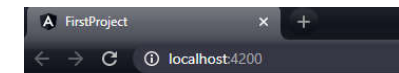
```
<> about.component.html X
src > app > about > <> about.component.html > div
1   <ul>
2     <li>Name : {{info.name}} </li>
3     <li>Email : {{info.email}} </li>
4     <li>Tel : {{info.tel}} </li>
5   </ul>
6   <div>
7     <input
8       type="text"
9       [(ngModel)]="comment.message">
10
11     <button
12       (click)="addComment()"
13       [disabled]="newComment">
14       Add Comment
15     </button>
16   </div>
17   <div *ngIf="comments.length>0 ; else noComments">
18     <h3>Messages</h3>
19     <ul>
20       <li *ngFor="let c of comments">
21         {{c.date | date}} : {{c.message}}
22       </li>
23     </ul>
24   </div>
25   <ng-template #noComments>
26     <p>No Comments</p>
27   </ng-template>
28 </div>
```



Data Binding : Tow Way Binding

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
8 export class AboutComponent implements OnInit {
9
10   info = {
11     name: "Anouar",
12     email: "medanouar.dahdeh@gmail.com",
13     tel: "12345678"
14   };
15   comments = [];
16   comment = { date: null, message: '' };
17   newComment = false;
18   constructor() { }
19
20   ngOnInit(): void {
21   }
22   addComment() {
23     if (this.comment.message !== '') {
24       this.comment.date = new Date();
25       this.comments.push({
26         date: this.comment.date,
27         message: this.comment.message
28       });
29       this.comment.message = '';
30     }
31   }
32 }
```

```
about.component.html X
src > app > about > about.component.html > div
1 <ul>
2   <li>Name : {{info.name}} </li>
3   <li>Email : {{info.email}} </li>
4   <li>Tel : {{info.tel}} </li>
5 </ul>
6 <div>
7   <input
8     type="text"
9     [(ngModel)]="comment.message">
10
11   <button
12     (click)="addComment()"
13     [disabled]="newComment">
14     Add Comment
15   </button>
16 </div>
17 <div *ngIf="comments.length>0 ; else noComments">
18   <h3>Messages</h3>
19   <ul>
20     <li *ngFor="let c of comments">
21       {{c.date | date}} : {{c.message}}
22     </li>
23   </ul>
24 </div>
25 <ng-template #noComments>
26   <p>No Comments</p>
27 </ng-template>
28
```



Welcome to first-project

- Name : Anouar
- Email : medanouar.dahdeh@gmail.com
- Tel : 12345678

 Add Comment

Messages

- Oct 16, 2020 : a first comment
- Oct 16, 2020 : a second comment
- Oct 16, 2020 : a third comment

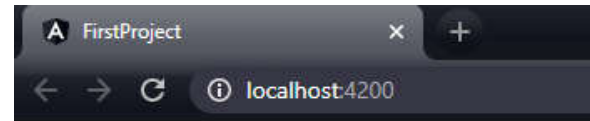
Importer **FormsModule** pour **ngModel**

- Pour utiliser la directive **ngModel**, il est nécessaire d'importer le module **FormsModule** de Angular dans **app.module.ts**

```
TS app.module.ts x
src > app > TS app.module.ts > ...
1  import { BrowserModule } from '@angular/platform-browser';
2  import { NgModule } from '@angular/core';
3  import { FormsModule } from '@angular/forms';
4  -----
5
6  import { AppRoutingModule } from './app-routing.module';
7  import { AppComponent } from './app.component';
8  import { AboutComponent } from './about/about.component';
9  import { ContactsComponent } from './contacts/contacts.component';
10
11  @NgModule({
12    declarations: [
13      AppComponent,
14      AboutComponent,
15      ContactsComponent
16    ],
17    imports: [
18      BrowserModule,
19      FormsModule,
20      AppRoutingModule
21    ],
22    providers: [],
23    bootstrap: [AppComponent]
24  })
25  export class AppModule { }
```

Exécution

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
8 export class AboutComponent implements OnInit {
9
10   info = {
11     name: "Anouar",
12     email: "medanouar.dahdeh@gmail.com",
13     tel: "12345678"
14   };
15
16   comments = [];
17
18   comment = { date: null, message: '' };
19
20   newComment = false
21
22   constructor() { }
23
24   ngOnInit(): void {
25   }
26
27   addComment() {
28     if (this.comment.message != '') {
29       this.comment.date = new Date();
30       this.comments.push({
31         date: this.comment.date,
32         message: this.comment.message
33       });
34       this.comment.message = '';
35     }
36   }
37 }
```



Welcome to first-project

- Name : Anouar
- Email : medanouar.dahdeh@gmail.com
- Tel : 12345678

No Comments

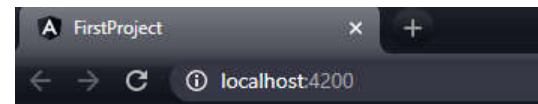
interpolation

NgModel

Property Binding

interpolation

Event binding



Welcome to first-project

- Name : Anouar
- Email : medanouar.dahdeh@gmail.com
- Tel : 12345678

Messages

- Oct 16, 2020 : a first comment
- Oct 16, 2020 : a second comment
- Oct 16, 2020 : a third comment

03

Bootstrap



Qu'est ce que Bootstrap ?



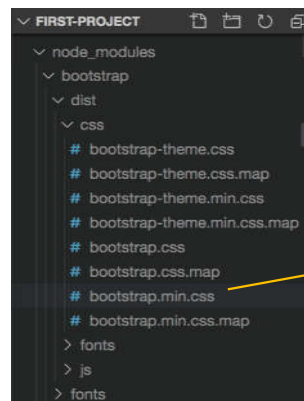
- **Les Framework CSS :**

<https://getbootstrap.com/>

- constituent en quelque sorte des boites à outils graphiques.
- sont extrêmement nombreux, parmi ceux-ci nous pourrions citer:
 - Tailwind CSS
 - Materialize CSS
 - **Bootstrap** (Le plus connu et le plus utilisé)
- **Bootstrap** est sous licence **open source** depuis 2011. Son **utilisation** est donc **gratuite**.
- Pour ajouter Bootstrap à un projet Angular, nous pouvons procéder de trois manières, en utilisant :
 - le fichier **angular.json**,
 - soit **index.html** ou bien le fichier **style.css**

Ajouter Bootstrap en utilisant angular.json

```
# 1- Installer la dernière version stable de Bootstrap
$ npm install --save bootstrap@4.6.2
# 2- Installer jQuery
$ npm install --save jquery
# 3- Ajouter les Chemins de bootstrap.min.css, jquery.js
# et bootstrap.js dans le fichier angular.json
# 4- Redémarrer le serveur
```



```
angular.json X
angular.json > {} projects > {} first-project > {} architect > {} build > {} configurations
newProjectRoot: projects,
projects: {
  "first-project": {
    "projectType": "application",
    "schematics": {},
    "root": "",
    "sourceRoot": "src",
    "prefix": "app",
    "architect": {
      "build": {
        "builder": "@angular-devkit/build-angular:browser",
        "options": {
          "outputPath": "dist/first-project",
          "index": "src/index.html",
          "main": "src/main.ts",
          "polyfills": "src/polyfills.ts",
          "tsConfig": "tsconfig.app.json",
          "aot": true,
          "assets": [
            "src/favicon.ico",
            "src/assets"
          ],
          "styles": [
            "./node_modules/bootstrap/dist/css/bootstrap.min.css",
            "src/styles.css"
          ],
          "scripts": [
            "./node_modules/jquery/dist/jquery.js",
            "./node_modules/bootstrap/dist/js/bootstrap.js"
          ]
        }
      }
    }
  }
}
```

Ajouter Bootstrap en utilisant index.html

```
# 1- Installer la dernière version stable de Bootstrap
$ npm install --save bootstrap@4.6.2
# 2- Installer jQuery
$ npm install --save jquery
# 3- Ajouter les Chemins de bootstrap.min.css, jquery.js
# et bootstrap.js dans le fichier index.html
# 4- Redémarrer le serveur
```



```
index.html X
src > index.html > ...
1 <!doctype html>
2 <html lang="en">
3 <head>
4   <meta charset="utf-8">
5   <title>FirstProject</title>
6   <base href="/">
7   <meta name="viewport" content="width=device-width, initial-scale=1">
8   <link rel="icon" type="image/x-icon" href="favicon.ico">
9   <link rel="stylesheet" href="../node_modules/bootstrap/dist/css/bootstrap.css">
10
11 </head>
12 <body>
13   <app-root></app-root>
14   <script src="../node_modules/jquery/dist/jquery.js"></script>
15   <script src="../node_modules/bootstrap/dist/js/bootstrap.js"></script>
16 </body>
17 </html>
```

Ajouter Bootstrap en utilisant style.css



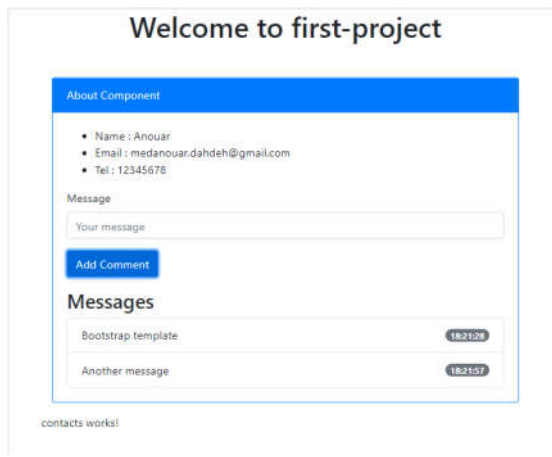
```
# 1- Installer la dernière version stable de Bootstrap
$ npm install --save bootstrap@4.6.2
# 2- Installer jQuery
$ npm install --save jquery
# 3- Importer bootstrap.css dans le fichier style.css @import "~bootstrap/dist/css/bootstrap.css"
# 4- Les fichiers .js seront importer en utilisant l'une des précédentes méthodes
# 5- Redémarrer le serveur
```



```
# styles.css M X
src > # styles.css
1  /* You can add global styles to this file, and also import other style files */
2  |  @import "~bootstrap/dist/css/bootstrap.css"
3
```

Nouveau Look avec Bootstrap

```
app.component.html X
src > app > app.component.html > ...
1 <div class="container">
2   <h1 class="text-center mb-5">Welcome to {{title}}</h1>
3   <app-about></app-about>
4   <app-contacts></app-contacts>
5 </div>
```



<https://getbootstrap.com/docs/4.5/components/>

©MAD

```
about.component.html X
src > app > about > about.component.html > div.container > div.card.border-primary.mb-3
1 <div class="container">
2   <div class="card border-primary mb-3">
3     <div class="card-header bg-primary text-white"> About Component</div>
4     <div class="card-body">
5       <ul>
6         <li>Name : {{info.name}} </li>
7         <li>Email : {{info.email}} </li>
8         <li>Tel : {{info.tel}} </li>
9       </ul>
10      <div class="form-group">
11        <label> Message</label>
12        <div class="input-group mb-3">
13          <input type="text" class="form-control" placeholder="Your message" [(ngModel)]="comment.message">
14        </div>
15        <button class="btn btn-primary" (click)="addComment()" [disabled]="newComment"> Add Comment</button>
16      </div>
17      <div *ngIf="comments.length>0 ; else noComments">
18        <h3>Messages</h3>
19        <ul class="list-group">
20          <li class="list-group-item d-flex justify-content-between align-items-center">
21            *ngFor="let c of comments">
22              {{c.message}} <span class="badge badge-secondary badge-pill">{{c.date | date:'HH:mm:ss'}}</span>
23          </li>
24        </ul>
25      </div>
26      <ng-template #noComments>
27        <p>No Comments</p>
28      </ng-template>
29    </div>
30  </div>
31 </div>
```

mohamedanouer.dahdeh@fsb.ucar.tn

04

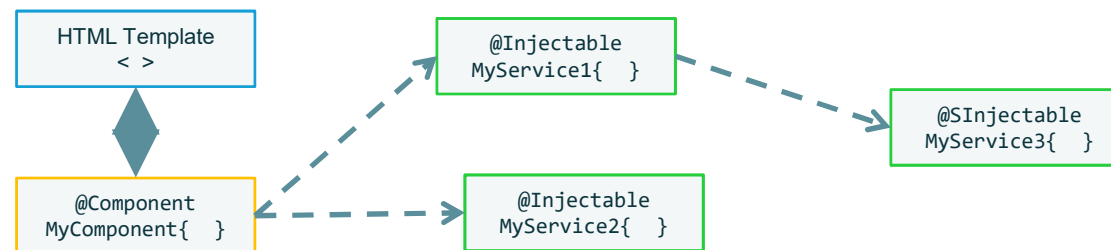
Services



Service (1/2)

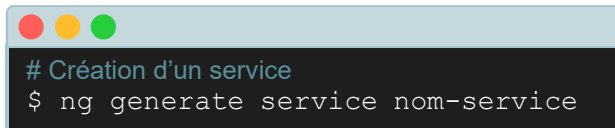
Objectifs

- Augmenter la **réutilisabilité** du code en séparant les fonctionnalités des composants des autres types de traitement
- Généralement :
 - **Les Composants** se limitent à l'affichage et à la gestion des événements utilisateurs dans les vues
 - **Services** : exécutent les traitements en local ou en back end
- A chaque événement, le composant fait appel à des fonctions dans les services.
- Les services interagissent avec le back end (via des requêtes HTTP).
- Les composants consomment les services (qui consomment d'autres services).
- L'utilisation d'un service se fait via le principe de **l'injection des dépendances**.



Service (2/2)

- Dans Angular, un service est :
 - une classe TypeScript décorée par `@Injectable`
 - souvent joue le rôle de l'**intermédiaire** avec la partie back-end
 - un moyen de communication entre composants
 - injectable dans les classes de composants où on a besoin de l'utiliser
- Quelques Services :
 - ActivatedRoute
 - Router
 - FormBuilder
 - HttpClient
- Pour créer un service



```
# Création d'un service
$ ng generate service nom-service
```

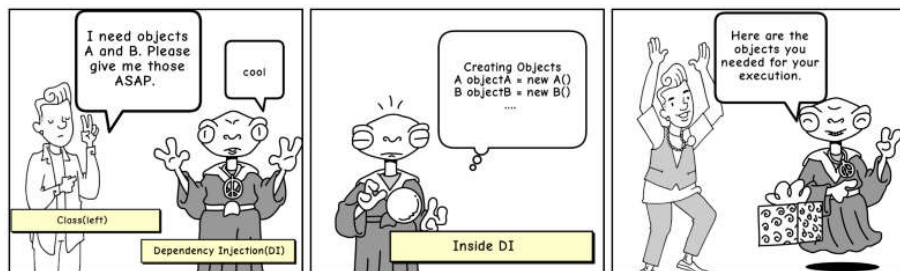
Comment utiliser un service ?



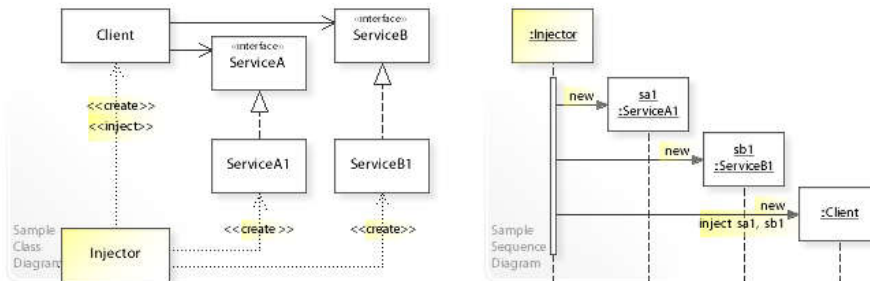
```
TS my-first.service.ts ×
src > app > services > TS my-first.service.ts > MyFirstService
1  import { Injectable } from "@angular/core";
2
3  @Injectable()
4  export class MyFirstService {
5
6      constructor(){}
7
8      saveData(date){
9          console.log('saving data at back end');
10     }
11
12     getDate(){
13         console.log('geting data from back end');
14     }
15 }
```

```
TS my-first.component.ts ×
src > app > my-first > TS my-first.component.ts > ...
1  import { Component, OnInit } from '@angular/core';
2  import { MyFirstService } from '../services/my-first.service';
3  ...
4  @Component({
5      selector: 'app-my-first',
6      templateUrl: './my-first.component.html',
7      styleUrls: ['./my-first.component.scss']
8  })
9  export class MyFirstComponent implements OnInit {
10
11      constructor(private myFirstService : MyFirstService) { }
12      ...
13      ngOnInit() {
14      }
15
16      onSaveDate(data){
17          this.myFirstService.saveData(data);
18      }
19
20      onGetData(){
21          this.myFirstService.getDate();
22      }
23 }
```


Principe de l'injection des dépendances



This comic was created at www.MakeBeliefsComix.com. Go there and make one now!



@Component

{constructor (Service)}

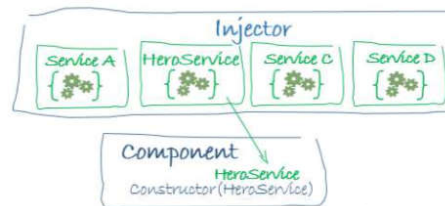
Service

Injection de dépendance

- Une façon de fournir une nouvelle instance d'une classe avec les dépendances
- La plupart des dépendances sont des services

Injection des dépendances dans Angular

- Au démarrage, Angular crée un injecteur (**Injector**) à l'échelle de l'application.
- L'injecteur crée des dépendances (le plus souvent sous forme de services) et maintient un conteneur d'instances de dépendances qu'il réutilise si possible.
- Lorsque Angular découvre qu'un composant dépend d'un service, il vérifie d'abord si l'injecteur a une instance existante de ce service. Si une instance du service demandé n'existe pas encore, l'injecteur en crée une à l'aide du provider enregistré et l'ajoute à l'injecteur avant de renvoyer le service à Angular.
- Lorsque tous les services demandés ont été résolus et renvoyés, Angular peut appeler le constructeur du composant avec ces services comme arguments.



Injection des dépendances dans Angular

- Les dépendances peuvent être fournies à trois niveaux :
 - **au niveau root:** c'est le comportement par défaut lors de la création d'un service avec le CLI. C'est ce que signifie `providedIn: 'root'`. La même instance de la dépendance est injectée partout où elle est nécessaire comme s'il s'agissait d'un singleton.
 - **au niveau du module:** la dépendance est ajoutée au tableau de providers du `NgModule`. Le module obtient sa propre instance de la dépendance
 - **au niveau du composant:** la dépendance est ajoutée au tableau des providers du composant. Chaque instance de ce composant obtient sa propre instance de la dépendance

Niveau root

```
1 import { Injectable } from '@angular/core';
2
3 @Injectable({
4   providedIn: 'root'
5 })
6 export class MyFirstService {
7
8   constructor() { }
9 }
```

Niveau du module

```
1 @NgModule({
2   //...
3   providers: [MyFirstService],
4   //...
5 })
6 export class AppModule { }
```

Niveau du composant

```
1 import { Component } from '@angular/core';
2 import { MyFirstService } from '../services/my-first.service';
3
4 @Component({
5   selector: 'app-my-first',
6   templateUrl: './my-first.component.html',
7   styleUrls: ['./my-first.component.css'],
8   providers: [MyFirstService]
9 })
10 export class MyFirstComponent { }
```

Revenons à notre exemple

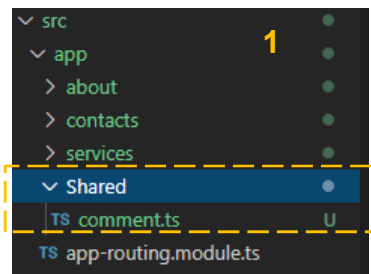
```
# Création du AboutService sous le dossier services
# shorthand for ng generate service services/about
$ ng g s services/about
```

```
TS about.service.ts X
src > app > services > TS about.service.ts > AboutService
1 import { Injectable } from '@angular/core';
2
3 @Injectable({
4   providedIn: 'root'
5 })
6 export class AboutService {
7   info = {
8     name: "Anouar",
9     email: "medanouar.dahdeh@gmail.com",
10    tel: "12345678"
11  };
12  comments = [];
13  constructor() { }
14  getInfos(){
15    return this.info;
16  }
17  addComment(c){
18    c.date = new Date();
19    this.comments.push(c);
20  }
21  getAllComments(){
22    return this.comments
23  }
24 }
```

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
1 import { Component, OnInit } from '@angular/core';
2 import { AboutService } from '../services/about.service';
3
4 @Component({
5   selector: 'app-about',
6   templateUrl: './about.component.html',
7   styleUrls: ['./about.component.css']
8 })
9 export class AboutComponent implements OnInit {
10
11   comment = {id:0, date:null, message:''};
12   newComment = false;
13
14   constructor(private aboutService : AboutService) { }
15   info = this.aboutService.getInfos();
16   comments = this.aboutService.getAllComments();
17
18   ngOnInit() {}
19
20   addComment(){
21     if (this.comment.message != ''){
22       this.aboutService.addComment({
23         message : this.comment.message
24       });
25       this.comment.message = '';
26     }
27   }
28 }
```

Ajout d'une classe modèle: Comment

```
# Création de la classe Comment sous le dossier shared
$ ng generate class shared/comment
```



```
TS comment.ts X
src > app > Shared > TS comment.ts > Comment
1 export class Comment {
2   id: Number;
3   date: Date = null;
4   message: string;
5 }
```

2

```
TS about.service.ts X
src > app > services > TS about.service.ts > ...
1 import { Injectable } from '@angular/core';
2 import { Comment } from '../shared/Comment';
3
4 @Injectable({
5   providedIn: 'root'
6 })
7 export class AboutService {
8   info = {
9     name: "Anouar",
10    email: "medanouar.dahdeh@gmail.com",
11    tel: "12345678"
12  };
13  //comments Array
14  comments: Comment []=[];
15
16  constructor() { }
17  getInfos(){
18    return this.info;
19  }
20  addComment(c:Comment){
21    c.date = new Date();
22    this.comments.push(c);
23  }
24  getAllComments(): Comment[] {
25    return this.comments
26  }
27 }
```

3

```
TS about.component.ts X
src > app > about > TS about.component.ts > ...
1 import { Component, OnInit } from '@angular/core';
2 import { AboutService } from '../services/about.service';
3 import { Comment } from '../shared/Comment';
4 @Component({
5   selector: 'app-about',
6   templateUrl: './about.component.html',
7   styleUrls: ['./about.component.css']
8 })
9 export class AboutComponent implements OnInit {
10   comment:Comment={ id: 0, date: null, message: '' };
11   newComment = false
12   info: any;
13   comments : Comment[];
14   constructor(private aboutService: AboutService) { }
15
16   ngOnInit(): void {
17     this.info = this.aboutService.getInfos();
18     this.comments = this.aboutService.getAllComments();
19   }
20
21   addComment() {
22     if (this.comment.message != '') {
23       this.aboutService.addComment({
24         id : this.comment.id,
25         date: this.comment.date,
26         message: this.comment.message
27       });
28       this.comment.message="";
29     }
30   }
31 }
```

4

Revisite de l'Architecture globale d'Angular

